CREATE: A Closed-Loop Reward Editing Agent with Training Evidence for Reinforcement Learning

Anonymous Authors
Anonymous Submission

***Abstract*—Reward engineering for reinforcement learning remains iterative and labor-intensive: large language models (LLMs) can generate reward programs, but a generated reward is only meaningfully evaluated after an expensive policy-training run, and most pipelines discard the resulting training evidence. This paper introduces CREATE, a closed-loop reward editing agent that treats each completed training run as a diagnostic observation rather than a scalar outcome. CREATE extracts component-level statistics to identify which reward semantic failed, applies a bounded single-semantic edit, and records the evidence–action–outcome chain in persistent memory while a best archive prevents regression. On LunarLander-v3, CREATE solves all five lineages under a ten-evaluation budget, whereas budget-matched independent generation and stateless revision solve none. Mechanism ablations confirm that enriched evidence and bounded editing are coupled requirements: removing either drops the solution rate to at most 2/5. A secondary study on BipedalWalker-v3 supports transfer to continuous-action locomotion. These results establish reward repair as an agentic closed-loop process in which failed training becomes actionable evidence.**

***Index Terms*—reinforcement learning, reward design, large language models, language agents, training evidence**

I. Introduction

Reinforcement learning (RL) depends critically on reward functions that express task intent while providing informative feedback for policy optimization [1]. Reward shaping can improve credit assignment, but only restricted transformations are guaranteed to preserve the optimal policy [2]. Inverse reward design further shows that a written reward should be treated as imperfect evidence of designer intent rather than as the intent itself [3]. Reward misspecification can therefore produce unsafe or unintended behavior even when the specified signal appears semantically reasonable [4].

Reward engineering is consequently an iterative debugging process rather than a one-shot coding task. Each completed training run produces more than a scalar return: episode length, termination patterns, reward-component activation, magnitude balance, and temporal trends can reveal how the reward program shaped the learned policy. Converting these heterogeneous records into a precise reward modification, however, still requires substantial human analysis.

Code-capable large language models (LLMs) can generate and iteratively refine reward programs from task descriptions and training feedback [5]–[11]. In parallel, LLMs are increasingly organized as agents that observe, reason, and act across trials, using stored feedback to improve later decisions [12]–[14]. These developments motivate an agent-oriented view of reward engineering in which policy training supplies observations and reward-code revision constitutes an external action.

This paper explores an agent-oriented view of reward engineering through CREATE, a *Closed-Loop Reward Editing Agent with Training Evidence*. CREATE places a reward-editing agent above an inner RL policy learner. After each policy-training run, a tool-using subagent compresses native-task outcomes, reward-component statistics, and training dynamics into a structured evidence summary. The reward-editing agent diagnoses one principal semantic failure, selects an edit level (L1/L2/L3), and applies a semantically localized repair. Intervention memory retains the evidence–action–outcome chain across rounds, while a separate best archive protects the strongest solution from later exploratory regressions.

Experiments on LunarLander-v3 compare CREATE against budget-matched independent generation and stateless revision, with two mechanism ablations isolating evidence enrichment and hierarchical semantic editing. BipedalWalker-v3 provides a secondary cross-task validation. Under a ten-evaluation budget, the experiments examine whether structured observation, persistent state, and bounded reward actions can convert failed policy training into reliable reward repair.

II. Related Work

A. Reward Engineering for Reinforcement Learning

Reward shaping augments task objectives with intermediate signals that improve credit assignment while preserving the optimal policy only under restricted transformations [2]. Inverse reward design treats a specified reward as imperfect evidence of designer intent and highlights the gap between written objectives and desired behavior [3]. More broadly, reward misspecification can produce unsafe or unintended behavior even when the reward appears semantically reasonable [4]. These results motivate methods that treat reward engineering as an iterative debugging problem rather than a one-shot specification step.

B. LLM-Based Reward Generation and Refinement

LLMs can map task descriptions to executable reward code. Language to Rewards demonstrates language-conditioned reward generation for robotic skill synthesis [5]. Text2Reward frames reward shaping as interpretable program synthesis from language instructions [6]. Eureka combines reward-code generation with evolutionary optimization and component-level

Code and supplementary material: https://github.com/Nicole-ying/expert-reward-agent

feedback [7]. CARD incorporates dynamic feedback into LLM-driven reward design [8]. REvolve studies reward evolution with human feedback in the loop [9]. A chain-of-thought reward-engineering approach shows that explicit reasoning traces can improve reward-design transparency [10]. Diagnostic-driven refinement treats reward failure as a debugging problem and uses training diagnostics for targeted revision [11]. Collectively, these methods show that training feedback supports reward revision beyond one-shot generation.

C. LLM Agents and Agentic Reward Engineering

Language-agent research organizes LLMs to maintain state and act in external processes. ReAct couples reasoning with tool-mediated actions [12]. Reflexion stores verbal feedback as episodic experience for later decisions [13]. Self-Refine shows how iterative self-feedback can improve generated outputs without parameter updates [14]. These mechanisms are entering reward design: RF-Agent formulates reward construction as sequential decision making [15], and STRIDE applies agentic reward engineering to humanoid locomotion [16]. CREATE follows this direction but centers the interaction on instrumented policy training: a diagnostic subagent converts training records into observations, and the reward-editing agent commits to a semantically localized edit measured by the next complete training run.

III. CREATE

CREATE comprises an outer reward-editing agent and an underlying RL agent. The RL agent interacts with the environment and learns a policy under the current generated reward, whereas CREATE treats each completed policy-training and native-evaluation run as one reward-design round. As shown in Fig. 1, tool-assisted evidence processing organizes the resulting records before the reward-editing agent diagnoses the current reward and executes a validated edit.

The resulting observe–reflect–act loop is written as

$$\begin{aligned} O_t &= \Phi_{\mathcal{T}}(\mathcal{D}_t), \\ (H_t, A_t) &= \Psi(R_t, O_t, M_t, B_t), \\ R_{t+1} &= \mathcal{U}(R_t, A_t), \end{aligned} \tag{1}$$

where the training record $\mathcal{D}_t$ is retrieved and compressed into observation O_t through evidence tools. This processing stage organizes information but does not select a reward edit. The reward-editing agent combines O_t with the current reward R_t, intervention memory M_t, and best archive B_t to produce reflection H_t, reward action A_t, and the next executable reward program R_{t+1}.

A. Task Understanding and Reward Initialization

Given the task description, environment interface, and reward-code constraints, CREATE identifies the principal behavior semantics and instantiates them as named reward components. The initial reward is

$$R_0(s, a, s') = \sum_{i=1}^{m_0} w_i^{(0)} r_i^{(0)}(s, a, s'), \tag{2}$$

where each component represents a behavior semantic such as task progress, stability, target events, or action cost. Named components are logged independently during training, enabling component-level diagnosis in later rounds. Generated code is checked for interface and execution validity before policy training; prompts and validation rules are provided in the supplementary material.

B. Training-Evidence-Driven Observe–Reflect–Act

At iteration t, the underlying RL agent optimizes the generated reward, while CREATE evaluates the learned policy with the environment-native objective:

$$\begin{aligned} \pi_t &= \arg\max_{\pi} \mathbb{E}_{\tau\sim\pi}\left[\sum_{k=0}^{H-1} \gamma^k R_t(s_k, a_k, s_{k+1})\right], \\ J_t &= \frac{1}{N}\sum_{n=1}^{N}\sum_{k=0}^{H_n-1} r_{\mathrm{env}}(s_k^{(n)}, a_k^{(n)}, s_{k+1}^{(n)}). \end{aligned} \tag{3}$$

Thus, the generated reward shapes policy learning, whereas the native objective measures task completion. Each run produces episode outcomes, termination patterns, component activation and scale, temporal dynamics, and differences from the preceding reward version. An auxiliary tool-using subagent retrieves and compresses these records into a structured evidence summary. It reports what occurred during training but does not recommend coefficient changes, select an intervention level, or modify reward code.

Component statistics serve as diagnostic cues rather than fixed editing rules. Persistent inactivity may indicate an unreachable gate or sparse formulation; excessive activation and magnitude may reveal a dominant proxy; frequent activation with weak contribution may indicate insufficient scale, saturation, or an ineffective mathematical form. The reward-editing agent combines these cues with policy outcomes, reward source code, and prior interventions to identify one principal reward semantic for the next edit:

$$A_t = (q_t, \ell_t, \Delta_t), \qquad \mathrm{supp}(\Delta_t) \subseteq \mathcal{C}(q_t), \tag{4}$$

where q_t denotes the selected semantic, ℓ_t the edit level, and $\mathcal{C}(q_t)$ the components and parameters implementing that semantic. An ordinary action may add or remove a component, adjust its weight or gate, or replace its mathematical form. Related components may be adjusted jointly when they express the same semantic, but independent design intents are not mixed within one ordinary revision. L1 calibrates coefficients, thresholds, or gates; L2 adds, removes, or refactors components while preserving the target semantic; and L3 replans the reward structure after repeated local failure.

C. Intervention Memory and Best-Reward Selection

After each round, CREATE records the observation, diagnosed semantic, reward action, expected change, and subsequent outcome in persistent intervention memory. This history supports comparison across rounds and discourages repeated unsuccessful edits. The best archive is maintained separately from the active lineage to prevent later regressions from

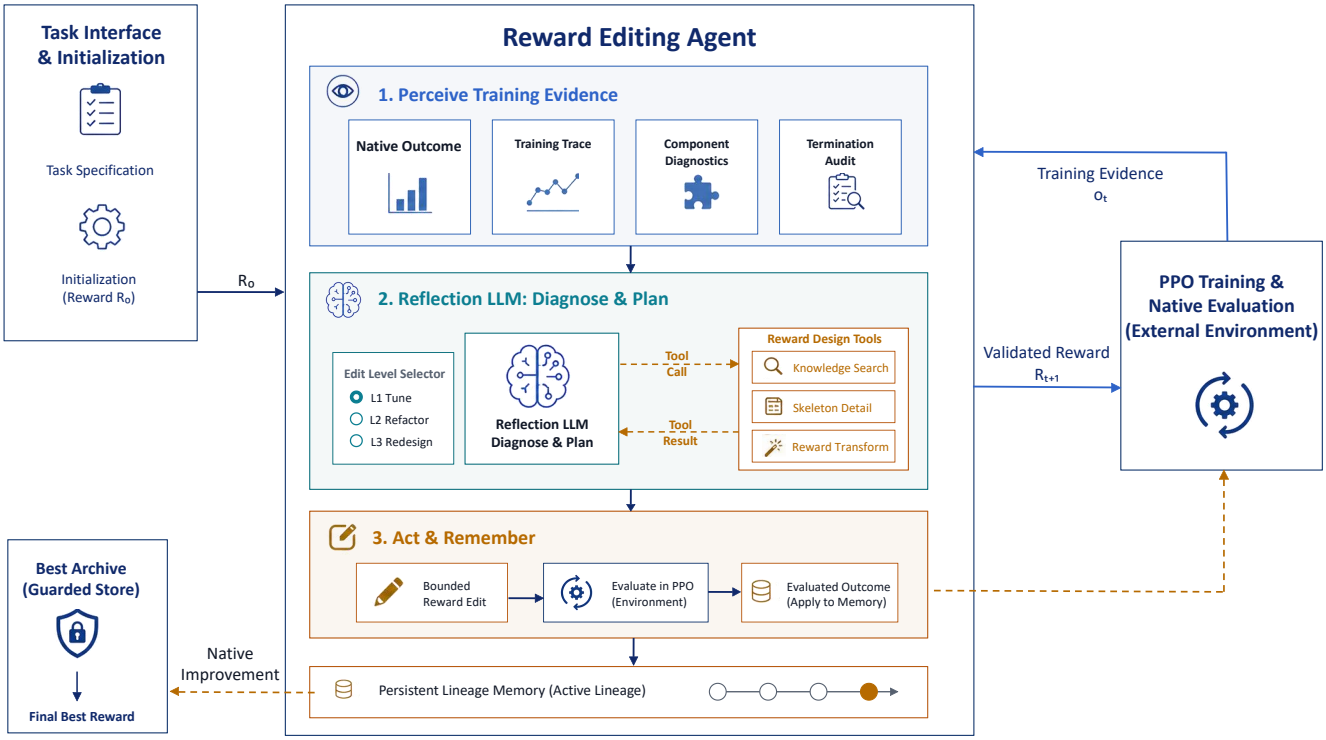

Fig. 1. Overview of CREATE. The underlying RL agent learns under the current generated reward. Tool-assisted evidence processing retrieves and compresses relevant training and lineage records, while the reward-editing agent remains responsible for failure diagnosis, semantic localization, intervention-level selection, and reward modification. Persistent memory records intervention outcomes, and the best archive retains the reward program with the highest native-task performance.

overwriting a previously discovered solution. After at most K reward evaluations, CREATE returns

$$t^* = \arg\max_{0 \le t < K} J_t, \qquad R^* = R_{t^*}. \tag{5}$$

The selected reward–policy pair is subsequently evaluated on independent episode seeds to assess stability.

D. Summary

Together, the three-stage architecture—evidence processing (Eq. 1), semantic diagnosis and bounded editing (Eq. 4), and intervention memory with best-reward selection (Eq. 5)—forms a closed diagnostic loop. Each completed policy-training run produces component-level evidence that reveals *which* semantic failed and *how*; the editor commits to a targeted intervention on that semantic alone; and the next training run measures whether the diagnosis was correct. Intervention memory preserves this chain across rounds, while the best archive guards against regression. The experiments in Section IV and V test whether this diagnostic architecture can convert failed policy training into reliable reward repair under a fixed evaluation budget.

IV. Experimental Procedure

This section details the experimental protocol that implements the CREATE framework described in Section III. We describe the setup, the five conditions under comparison, and the training and evaluation pipeline. Results are deferred to Section V.

A. Experimental Setup and Configuration

Experiments are conducted on two Gymnasium environments [17]. LunarLander-v3 serves as the primary benchmark for baseline comparison and mechanism ablation; BipedalWalker-v3 provides a secondary validation on continuous-action locomotion. Every reward program trains a freshly initialized proximal policy optimization (PPO) policy [18] through Stable-Baselines3 [19]. LunarLander uses 10^6 environment steps per candidate and BipedalWalker uses 5×10^6. DeepSeek V4 Pro serves as the LLM backbone for all LunarLander conditions. Full prompts, model settings, hyperparameters, per-seed traces, and detailed environment configurations are provided in the supplementary material.

B. Condition Definitions

Five conditions are compared on LunarLander-v3 under a shared LLM backbone, environment description, reward interface, code checks, and PPO budget. Each condition contains five independent reward-search lineages and at most ten complete reward evaluations, so that every condition operates under the same total policy-training cost. All four iterative conditions (Stateless revision, CREATE without evidence enrichment, CREATE without hierarchical editing, and CREATE) share a common initial reward produced by CREATE's task-understanding phase (Section III-A); the single-shot evaluation of this reward yields a mean fitness of -2.21 ± 73.38 (0/5

solved), confirming that an initially plausible reward is not reliably learnable without subsequent refinement.

Independent-10 serves as a budget-matched search-breadth control: it generates ten unrelated initial reward candidates, each from an independent invocation of the reward-initialization phase, and evaluates each with a separate policy-training run. The best candidate is selected post hoc. This condition tests whether evaluating ten unrelated candidates, without any cross-round information flow, can match CREATE's iterative diagnostic process.

Stateless revision retains the sequential LLM invocation of CREATE but removes the evidence organizer, intervention memory, cross-round component comparison, and hierarchical semantic editing. The LLM receives only basic current-round feedback and may modify any number of components simultaneously, with no constraint on edit scope. This condition serves as the baseline against which CREATE's architectural contributions are measured.

CREATE without evidence enrichment retains the full loop, memory, best archive, and hierarchical editing, but removes the tool-using evidence organizer: the editor receives native fitness, episode length, termination statistics, and component episode sums, but lacks activation rates, magnitude shares, temporal trends, and cross-round summaries.

CREATE without hierarchical editing retains enriched evidence, memory, and the best archive, but removes the semantic-localization constraint (Eq. 4), allowing unconstrained multi-component rewrites.

CREATE runs the complete architecture described in Section III.

C. Training and Evaluation Protocol

Each reward-design round consists of a complete PPO training run followed by native evaluation. The policy is optimized with the generated reward R_t, whereas candidate quality is measured only by the untouched environment reward r_{env} (Eq. 3). We define *search fitness* as the mean undiscounted native episodic return over 20 fixed evaluation episodes and *best search fitness* as the largest value retained within a lineage. After reward selection, the returned policy is evaluated over 100 unseen episodes; this is reported as *test fitness*. The task-solving thresholds are 200 for LunarLander and 300 for BipedalWalker.

Between rounds, the evidence organizer (Eq. 1, $\Phi_{\mathcal{T}}$) compresses training records into a structured summary. The reward-editing agent then selects one principal semantic, chooses an edit level (L1/L2/L3), and applies a bounded modification (Eq. 4). The modified program is validated before the next round. The best archive is updated only when a new candidate exceeds the current incumbent in native search fitness (Eq. 5). On BipedalWalker-v3, only the complete CREATE workflow is evaluated; no baseline or ablation comparison is conducted on that task.

TABLE I
LUNARLANDER-V3 REWARD SEARCH OVER FIVE LINEAGES. VALUES ARE MEAN ± SAMPLE STANDARD DEVIATION OF LINEAGE-WISE BEST SEARCH FITNESS.

Condition	Budget	Best fitness	Solved
Independent-10	10	-0.74 ± 114.40	0/5
Stateless revision	10	44.0 ± 65.9	0/5
CREATE w/o evidence enrichment	10	134.97 ± 148.43	2/5
CREATE w/o hierarchical editing	10	114.21 ± 46.31	0/5
CREATE	10	$\mathbf{228.98 \pm 16.54}$	**5/5**

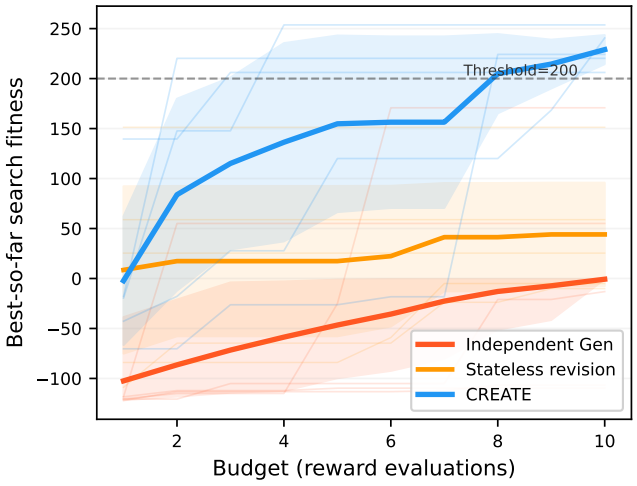

Fig. 2. Best-so-far LunarLander search fitness under a matched policy-training budget. Each curve averages five lineages after retaining the incumbent within each lineage; the dashed line denotes the task-solving threshold.

V. RESULTS AND ANALYSIS

A. Reward Search Under a Matched Training Budget

Search breadth alone does not produce reliable reward repair. Table I compares five conditions under identical policy-training budgets. Independent-10 evaluates ten unrelated candidates without cross-round information flow, yet solves no lineage—ruling out that candidate count alone drives improvement. The four iterative conditions share a common initial reward; subsequent differences therefore reflect each architecture's capacity to convert training outcomes into better reward functions.

Iterative rewriting without structured evidence or memory produces non-monotonic progress. Stateless revision retains sequential LLM invocation but removes evidence enrichment, memory, and editing constraints. It fails to solve any lineage (Table I), and in three of five seeds the initial reward already outperforms every subsequent revision. Without component-level diagnostics or memory, the LLM cannot distinguish a poorly calibrated coefficient from a fundamentally wrong semantic—each rewrite is effectively a guess. CREATE, by contrast, solves every lineage, demonstrating that closed-loop evidence, not repeated invocation, drives reliable repair.

Fig. 2 reveals the qualitative gap: independent generation improves only by chance; stateless revision plateaus despite repeated rewriting; CREATE progressively raises its incumbent

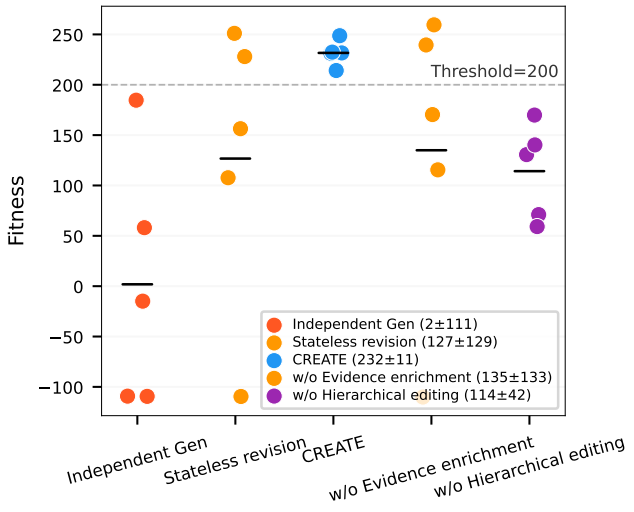

Fig. 3. Per-seed held-out test fitness and mechanism ablation results on LunarLander. Horizontal bars mark group means; the dashed line denotes the task-solving threshold (200).

across all lineages. The key distinction is whether each training run becomes diagnostic evidence rather than a binary outcome. Held-out fitness (Fig. 3) confirms the advantage persists under unseen evaluation seeds.

B. Mechanism Ablations

We now isolate *why* the closed loop works by testing two mechanism hypotheses from Section III.

Enriched evidence enables diagnosis. Activation rates, magnitude shares, and temporal trends enable the editor to distinguish mechanistically distinct failure modes—for instance, an inactive component could be unreachable, too weak, or behaviorally irrelevant, and a scalar sum cannot tell them apart.

Without the evidence organizer, the editor receives only scalar outcomes (fitness, episode length, termination statistics, component sums). This variant solves only two of five lineages with high dispersion (Table I), and one lineage remains negative throughout. Aggregate scores can support occasional repair but cannot distinguish *why* a component failed: *what* evidence is provided determines *which* diagnoses are possible.

Hierarchical editing enables single-trial attribution. The semantic-localization constraint (Eq. 4) ensures each retraining run serves as a controlled test of one intervention. Without it, multi-component edits prevent attributing a performance change to any specific modification.

Without hierarchical editing, the editor retains enriched evidence and memory but applies unconstrained multi-component rewrites. This variant solves no lineage (Table I), and one seed collapses catastrophically. When several unrelated components change together, one training result cannot identify which intervention caused the outcome—each 10^6-step run becomes an uncontrolled experiment rather than a focused test. Bounded action is necessary for single-trial attribution.

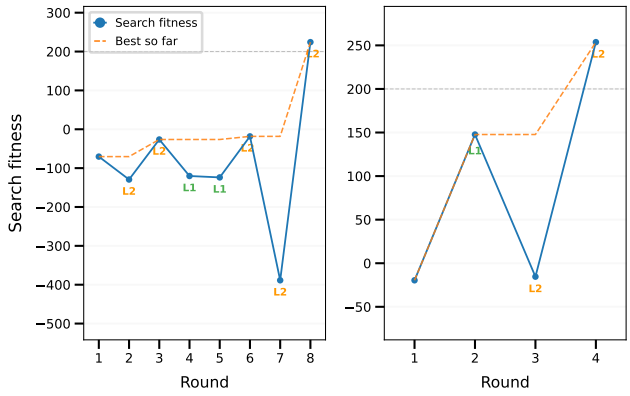

Fig. 4. Two representative CREATE repair trajectories on LunarLander. Solid markers show search fitness per round; the dashed line tracks the best-so-far incumbent. L1/L2 labels mark the edit level at each intervention.

Fig. 3 shows that enriched evidence and bounded editing are coupled requirements. Evidence-poor repair occasionally succeeds but is initialization-dependent; unconstrained editing has lower dispersion only because all lineages remain below the threshold. Full CREATE combines 5/5 solutions with a substantially tighter distribution (see Table I and Fig. 3).

C. How Evidence Becomes an Edit

We now examine *how* component evidence is translated into edits, through two representative CREATE lineages (Fig. 4; heatmaps in supplementary material).

Collapse and recovery (seed 0). In round 5, the evidence organizer reports that a distance-reduction signal is 100% active yet all episodes crash within 68 steps—the agent dives at fatal speed. The editor diagnoses proxy misalignment and plans an L2 intervention: replace the delta-based signal with state-based proximity plus descent safety and landing bonuses. Round 7 executes this plan but collapses to -389: the new approach-shaping signal is net negative, providing no gradient toward the pad, and the landing bonuses record 0% activation. Intervention memory preserves the round 6 solution (fitness -18.28), which used state-based proximity and reached the pad. In round 8, the editor compares across rounds—state-based proximity reached the pad; delta-based shaping collapsed—and applies an L2 edit restoring state-based proximity with an anti-farming height factor. Fitness rises to 224, and the best archive guards this solution when round 9's exploratory edit fails.

Progressive convergence (seed 3). Seed 3 improves from -20 to 148 after an L2 refactor replaces an absolute-distance penalty with potential-difference shaping, since the distance term provided a constant penalty regardless of direction. A subsequent overly restrictive adjustment causes a regression. Intervention memory localizes the cause: the pre-regression version achieved 148, so the regression stems from the most recent change. The next L2 edit reconstructs the contact semantic as a reachable gradient, and fitness reaches 254. Hierarchical editing prevents simultaneous modification of multiple semantics, making single-trial attribution possible.

TABLE II
BIPEDALWALKER-V3 RESULTS FOR COMPLETE CREATE ONLY. v DENOTES THE EVALUATED REWARD VERSION.

Seed	Initial fitness	First $v \geq 300$	Best fitness
0	270.7	v_2	320.0
1	280.5	v_2	313.7
2	272.1	v_2	307.9
3	290.0	v_2	311.1
4	103.0	v_3	304.9
Mean	243.26 ± 70.45	2.2 versions	311.54 ± 5.17

Across both trajectories, component evidence identifies a principal semantic failure, the editor applies a targeted revision, and intervention memory preserves the best solution for cross-round comparison.

D. Cross-Task Validation

BipedalWalker-v3 validates CREATE on continuous-action locomotion. All five lineages cross the threshold within three evaluations (Table II); selected policies obtain 310.82 ± 5.03 test fitness. Mechanism ablations are reserved for LunarLander-v3 (Section V-B).

In summary, evidence without editing provides diagnosis without attribution; editing without evidence constrains action without guiding it.

VI. CONCLUSION

This study presented CREATE, a closed-loop reward editing agent that treats each policy-training run as diagnostic evidence rather than a binary evaluation. On LunarLander-v3, CREATE solved all five lineages under a ten-evaluation budget, whereas budget-matched independent generation and stateless revision both solved zero. Mechanism ablations demonstrated that enriched evidence and hierarchical semantic editing are coupled requirements: enriched evidence enables the editor to distinguish mechanistically distinct failure modes that scalar outcomes cannot differentiate, while bounded editing ensures each retraining run serves as a controlled attribution test. Cross-task validation on BipedalWalker-v3 provided supporting evidence of transfer to continuous-action locomotion. While these results establish the diagnostic closed-loop architecture as an effective paradigm for reward repair, the evidence-enrichment design was developed and ablated exclusively on LunarLander; its transfer to substantially different state-action structures remains an open question. Future work should examine evidence-organizer design for broader domain coverage, robustness to training-instability noise, and integration with stronger policy optimization methods.

REFERENCES

[1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

[2] A. Y. Ng, D. Harada, and S. Russell, "Policy invariance under reward transformations: Theory and application to reward shaping," in *Proceedings of the 16th International Conference on Machine Learning*, 1999, pp. 278–287.

[3] D. Hadfield-Menell, S. Milli, P. Abbeel, S. Russell, and A. Dragan, "Inverse reward design," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[4] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané, "Concrete problems in AI safety," *arXiv preprint arXiv:1606.06565*, 2016.

[5] W. Yu, N. Gileadi, C. Fu, S. Kirmani, K.-H. Lee, M. G. Arenas, H.-T. L. Chiang, T. Erez, L. Hasenclever, J. Humplik, B. Ichter, T. Xiao, P. Xu, A. Zeng, T. Zhang, N. Heess, D. Sadigh, J. Tan, Y. Tassa, and F. Xia, "Language to rewards for robotic skill synthesis," in *Proceedings of the 7th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 229, 2023, pp. 374–404.

[6] T. Xie, S. Zhao, C. H. Wu, Y. Liu, Q. Luo, V. Zhong, Y. Yang, and T. Yu, "Text2reward: Reward shaping with language models for reinforcement learning," in *International Conference on Learning Representations*, 2024.

[7] Y. J. Ma, W. Liang, G. Wang, D.-A. Huang, O. Bastani, D. Jayaraman, Y. Zhu, L. Fan, and A. Anandkumar, "Eureka: Human-level reward design via coding large language models," in *International Conference on Learning Representations*, 2024.

[8] S. Sun, R. Liu, J. Lyu, J.-W. Yang, L. Zhang, and X. Li, "A large language model-driven reward design framework via dynamic feedback for reinforcement learning," *Knowledge-Based Systems*, vol. 326, p. 114065, 2025.

[9] R. Hazra, A. Sygkounas, A. Persson, A. Loutfi, and P. Z. D. Martires, "REvolve: Reward evolution with large language models using human feedback," in *International Conference on Learning Representations*, 2025.

[10] X. Zhu, J. Du, Q. Fu, and L. Chen, "LLM-based reward engineering for reinforcement learning: A chain of thought approach," in *2025 10th International Conference on Cloud Computing and Big Data Analytics (ICCCBDA)*, 2025.

[11] Y. Wang, Y. Tang, B. Liu, X. Liu, and D. Shang, "When LLM reward design fails: Diagnostic-driven refinement for sparse structured RL," *arXiv preprint arXiv:2605.28918*, 2026.

[12] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations*, 2023.

[13] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[14] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark, "Self-refine: Iterative refinement with self-feedback," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[15] N. Gao, X. Zhang, X. Jiang, M. You, M. Zhang, and Y. Deng, "RF-Agent: Automated reward function design via language agent tree search," in *Advances in Neural Information Processing Systems*, vol. 38, 2025.

[16] Z. Wu, J. Lu, Y. Chen, Y. Liu, Y. Zhuang, and L. Hu, "STRIDE: Automating reward design, deep reinforcement learning training and feedback optimization in humanoid robotics locomotion," *arXiv preprint arXiv:2502.04692*, 2025.

[17] M. Towers, A. Kwiatkowski, J. Terry, J. U. Balis, G. D. Cola, T. Deleu, M. Goulão, A. Kallinteris, M. Krimmel, A. KG, R. Perez-Vicente, A. Pierré, S. Schulhoff, J. J. Tai, H. Tan, and O. G. Younis, "Gymnasium: A standard interface for reinforcement learning environments," *arXiv preprint arXiv:2407.17032*, 2024.

[18] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[19] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.